An Evaluation of The Performance of Lightweight Language Models on Zero-Shot NL2SQL tasks

Tom Fan
Carleton University
Ottawa, Canada
tomfan3@cmail.carleton.ca

1 ABSTRACT

Since their introduction as a consumer product in 2022, transformer-based large language models have been used for automating many kinds of digital tasks. This includes natural language to code and SQL tasks, which have exploded in research and development in the last few years. While a large proportion of LLM use is likely done using proprietary, closed source, and large footprint foundation models hosted by SAAS providers like Anthropic or OpenAI, an increasing amount of open-source models are also entering the market. These models have demonstrated promise on several different kinds of tasks, such as code generation using Python [8]. However, there is limited research on how they perform in other common coding-adjacent tasks, including NL2SQL. While there is an emerging wealth of assessment on how larger models like OpenAI's closed source GPT series perform on NL2SQL tasks in various configurations [10, 12], the performance of these small models' smaller cousins in non-pretrained configurations is still an underexplored area. In this paper, we aim to provide an overview of how several such opensource models perform when faced with NL2SQL tasks on the Spider-Dev question dataset.

2 INTRODUCTION

In recent years, there has been a significant amount of high performing open source LLMs available from both research institutions and tech companies [13]. While many of the top-ranking models require expensive and complex hardware setups to run as their parameter counts run in the hundreds of billions or even trillions, many are either small footprint by design or distilled versions of larger source models.

In these cases, consumer level hardware should be able to run the models using tools like localLLama, enabling their use offline and without dependency on external SAAS providers (for purposes of this paper, we define consumer level hardware to be current generation Nvidia RTX chips or other equivalents, able to comfortably run models in the 20B parameter range or smaller at 8 or 16b quantization).

These LLMs have shown promise in their ability to solve an array of tasks, including completing test sets for code generation such as HumanEval [6, 8]. In these experiments, the authors were able to push these models to perform disproportinately well to how large their footprints are, indicating that these smaller LLMs are able to consistently solve simple, well defined tasks and generate effective code.

At the same time, there has been significant development in the field of NL2SQL tools. While previous generations of NL2SQL tools depended on hardcoded rules or deep neural networks [10], the emergence of LLMs has enabled a whole new paradigm of NL2SQL tools based on them. In fact, in their MVP state, a LLM can serve as a NL2SQL tool on its own.

The applications of NL2SQL are many, ranging from business intelligence dashboards that allow non-technical users to query corporate databases, to research tools that enable scientists to explore datasets without SQL expertise, to customer support systems that can retrieve information from product databases through natural language queries.

Locally runnable NL2SQL tools offer even greater potential, enabling privacy-sensitive applications like healthcare data, financial regulation compliant reporting, and internal business analytics where data cannot leave a set premises due to regulatory or competitive concerns. They also can avoid reliance on external providers, preventing vendor lock in, service uptime issues, poor internet connections, and mitigating the impact of pricing changes. In this interest, the aim of this paper is to explore the potential of using small, opensource LLMs that can be run locally as NL2SQL generation tools.

While there are many evaluations of the potential of LLMs for NL2SQL tasks, many of these depend on the aforementioned large, closed-source, proprietary models [10, 12, 17]. At the same time, evaluations of the performance of lightweight, open source models are few in number. Therefore, in this paper we conduct an evaluation asking one guiding question - how well do lightweight LLMs such as Llama 8b or Qwen3 perform when presented with NL2SQL tasks in a zeroshot format?

To answer this guiding question, we took a sample of 8 such models in the 0-20b parameter range and evaluated their performance on the SPIDER-dev NL2SQL question set. Using a zero-shot method with a short context length, we built a testbench [18] to evaluate these opensource models using Novita's online AI service and hardware rental service [2]. This allowed us to run each model on the cloud with consistent hardware, while still being able to evaluate token efficiency and getting results equivalent to what we would see running these models on a local system.

Then, we gave each model we evaluated all 1034 of the SPIDER-dev NL2SQL questions, and recorded their responses. Models were not given pre-training or any other context.We found that while

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

all of these models can zero-shot these NL2SQL problems roughly $\frac{1}{2}$ to $\frac{2}{3}$ of the time in terms of execution accuracy, current small opensource LLMs are still not powerful enough to allow this type of use case consistently.

3 TESTING METHODOLOGY AND EXPERIMENTAL DESIGN

We started our experiment with the objective of evaluating the performance of lightweight open source models on NL2SQL tasks. In order to do so, we used SPIDER-dev as our test set (taoyds/spider) [19], which is a commonly used benchmark for evaluation of NL2SQL tools. Then, we evaluated the extracted SQL from the LLMs' responses using the accompanying taoyds/test-suite-sql-eval tool [21], as per the recommendations of SPIDER's authors [19]. To support these requirements, we built a testbench[18] that would serve as a bridge and data recording tool for SPIDER, the evaluator, as well as Novita, our LLM provider.

We chose to use Novita as an inference provider as it has a variety of features that were useful for this paper. Novita provides a wide array of opensource models in their original configuration compared to Groq, and provides the ability to rent consumer grade Nvidia GPUs unlike HuggingFace, which does not provide this type of GPU to use for inference.

While the general architecture of our testing system is fairly straightforward, tradeoffs had to be made in experiental design in several areas. These primarily happened in terms of model selection, test set selection, and prompting strategy and arose from a need for testing consistency and efficiency.

Model selection

Since this paper focuses on the use of small, opensource models with NL2SQL tasks, model selection was one of the most important design choices we made while determining the parameters of our experiments. Our experimental data incorporates testing from 8 different open source LLMs, each with a parameter count $\leq$ 30b. The basic criteria for evaluation included the following requirements:

- Models must be open source, or otherwise downloadable and usable for a user locally at no charge and with no external services. This constrains the selected models to effectively what can be found on HuggingFace.
- Models cannot be fine-tuned specifically for NL2SQL tasks, as this would go against the core evaluation of small, general purpose models being used for NL2SQL tasks. Other fine tuning, however, would not be a deal breaking factor for selection.
- Models must be able to run locally on consumer level hardware. While that which constitutes the category of consumer level hardware is subjective to the end user, for the purposes of this paper we have defined it as current generation consumer CPUs (Intel i series, AMD Ryzen, etc) alongside current generation Nvidia RTX graphics processors or equivalent. This constrains the size of the model to 20-30GB total, as most local LLMs will require $\geq$ 2GB of memory for KV cache or buffer space.

Using this selection criteria, we selected and tested a list of these 8 models available publically:

- llama 3.2 3b instruct [15]
- Qwen 3 4b [4]
- Qwen 2.5 7b instruct [3]
- llama 3.1 8b [14]
- Qwen 3 8b [5]
- Deepseek R1 distill 14b [1]
- GPT OSS 20b [16]
- Gemma 3 27b [7]

At either 8 or 16b quantization, these models should be able to fit in the 32GB of VRAM provided by an RTX 5090, the current flagship model of Nvidia's consumer GPU lineup. These models come from a variety of different developers, have a wide range of parameter counts, and an assortment of architectures (MOE, distilled, etc).

Although we did tests for Qwen's 30b coder and 32b models, these were excluded from testing as a VM instance running what we defined as "consumer level hardware" was not able to produce responses efficiently using these two models as-is.

Additionally, we ran the same tests using Google's Gemma3-12b-it model. We chose to exclude this from testing as it delivered parsable SQL only 68% of the time, and its results effectively broke the NL2SQL response tester. This leaves it with a score of 0 in every category in our tests, and did not deliver any meaningful insights.

Test set selection

For this paper, we selected the SPIDER-dev [19] NL2SQL dataset as the test set for our models. This was selected from a list of candidates including SPIDER-dev, SPIDER, SPIDER2, and BIRD. Even though these are all well designed, commonly used NL2SQL question sets, we opted to use solely SPIDER-dev for several key reasons.

The full SPIDER dataset contains 10,181 questions [20]. Even though we used a cloud provider to accelerate our inference with better hardware vs running locally, each run of the 1,034 questions in the SPIDER-dev dataset took over 12 hours on average. For a dataset 10x larger, this means that all 9 models would have taken over a month to return a full set of results, from running inference to extracting and running SQL, to recording the results to persistent storage.

The SPIDER2 dataset [9] also contains 10,000+ questions, as well as a significantly higher level of question difficulty. In preliminary testing, the smallest of the models quickly ran out of context window and were therefore unable to deliver a complete SQL query, both breaking the test platform and effectively eliminating this dataset from consideration. Similar issues exist in the BIRD dataset as well [11].

Thus by elimination, the SPIDER-dev question dataset was the last remaining contender under consideration. It contains a variety of databases, question types, and difficulty levels, meaning that even though it is significantly smaller in scope to the other datasets considered, testing a model using it would still lead to meaningful insights into how they performed in different scenarios. Additionally, since an already existing query evaluation tool was written for the SPIDER questions[21], we could leverage it to only need to store our LLM responses and utilize the output from the tool itself.

Prompting Strategy

Every benchmark of LLMs on any task requires a prompting strategy. This must be unified across all models evaluated, since we want all variables besides the model type to be as consistent as possible. Since our goal was to evaluate zero-shot performance on these models, we aimed to create a single text prompt that had to include both the SPIDER-dev question as well as any schema needed for context.

Initially, we took a multi-stage prompting approach where models were given a chain of thought and prompted to:

(1) read the schema and context given
(2) sort out what components of the schema will be relevant to the question
(3) build a SQL query based off the question
(4) reevaluate the produced query for potential errors
(5) return a single SQL query string based on SQLite's SQL dialect

However, this process would sometimes exceed the token window of the smaller models like Llama 3b or Qwen 4b, resulting in malformed returned strings, partial results, or other errors that made the results effectively unusable. To allow for one consistent prompting strategy across all models regardless of size, we opted to use a very simple prompt, shown below. All prompts were run with a temperature of 0.1, and a max token window of 8,192 in order to stay within budget and below Novita's API rate limit.

```
You are an expert SQL analyst.
Generate a SQL query to answer the
question based on the provided
database schema.

<CONTEXT/SCHEMA>

Question: <QUESTION - FROM
SPIDER-DEV DATASET>.

Requirements:
- Use proper SQL syntax for SQLite
- Return ONLY the final SQL query
- Do NOT include any explanations,
  reasoning, or additional text
- Do NOT wrap the query in backticks,
  code blocks, or markdown formatting
- Do NOT include prefixes like "sql:",
  "query:", or similar

SQL Query:
```

Figure 1: Zero-shot prompting template used for model tests

Using this prompt, we were able to extract usable SQL the vast majority of the time from our models. Even though the prompt instructed the model to return only the SQL query, some models were not able to follow this instruction. As such we added a REGEX based extraction feature to our testbench, which was able to extract SQL when the model returned a query string in most cases, such as when the string is preceded by a newline character, or when it is formatted as SQL:<query>.

With these three design choices in mind, we were able to use Novita AI's cloud AI service to evaluate our selected models on the SPIDER-dev dataset, we were able to run our desired tests and gather some meaningful insights into how the models behave when faced with a variety of NL2SQL tasks.

4 RESULTS

In testing, each of the selected models was faced with all 1,034 of the SPIDER-dev dataset questions. We measured three primary metrics - completion rate (whether or not the model returned parsable SQL), test accuracy (predicted SQL successfully completing the query correctly), and exact match accuracy (predicted SQL perfectly matching the benchmark SQL string).

Completion Rates

All models tested were able to complete the vast majority of prompts, with most models returning parsable SQL more than 98% of the time.

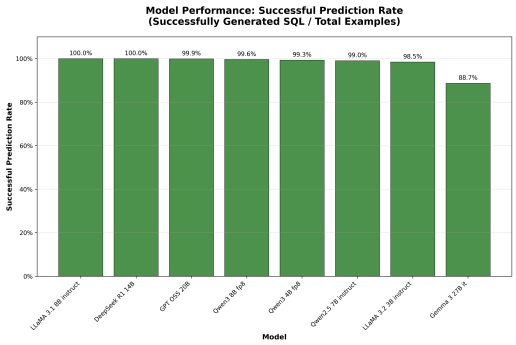

Figure 2: Bar chart of successful SQL generation rates per model

The exception to this was Google's Gemma-3 27b model, which returned parsable SQL only 88% of the time. It is notable that both Gemma-3 models failed to return parsable SQL at anything close to the rate at which the other tested models did.

Execution Accuracy

During testing, we saw significantly more variation in terms of execution accuracy across all assessed models than in the proportion of questions that resulted in parsable SQL.

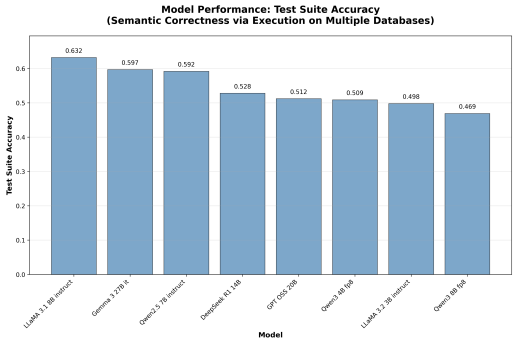

Figure 3: Bar chart of execution accuracy for different models assessed in this paper

The execution accuracy rate ranges from 63% on the high end with Meta's Llama 8b instruct to 46% on the low end with Alibaba's Qwen3 8b. Across all models, this left us a mean accuracy of 54.2%.

Across the different question difficulties, we also saw a variation in execution accuracy.

Model	Easy	Medium	Hard	Extra
Llama 3.1 8b instruct	0.835	0.684	0.471	0.361
Gemma 3 27b it	0.790	0.601	0.552	0.343
Qwen 2.5 7b instruct	0.831	0.587	0.511	0.331
Deepseek R1 14b	0.879	0.590	0.282	0.096
GPT OSS 20b	0.863	0.460	0.466	0.175
Qwen 3 4b	0.855	0.540	0.287	0.139
Llama 3.2 3b instruct	0.653	0.545	0.385	0.259
Qwen 3 8b	0.810	0.509	0.276	0.054

Most models were overwhelmingly successful on the easiest question difficulties, but struggled greatly on the extra hard questions. The exception to this rule was Llama-3.2-3b-instruct, which while completing only 65% of easy questions successfully, completed almost 26% of the hardest category questions. This is especially impressive considering that is the smallest footprint model we assessed in this paper.

Exact Match Accuracy

While the majority of assessed models managed execution accuracy on a majority of the SPIDER-dev tests, the models struggled significantly more on exact match accuracy metrics.

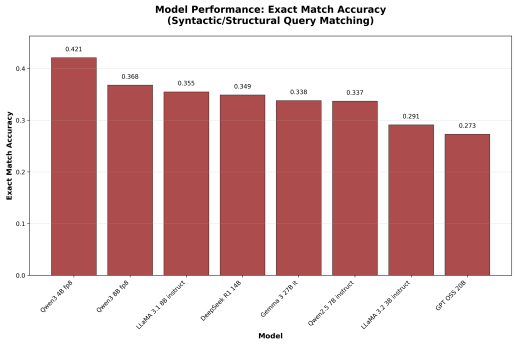

Figure 4: Bar chart of exact match accuracy for different models assessed in this paper

Model	Easy	Medium	Hard	Extra
Qwen 3 8b	0.746	0.359	0.172	0.030
Deepseek R1 14b	0.714	0.352	0.115	0.042
Qwen 2.5 7b instruct	0.661	0.283	0.230	0.108
Llama 3.1 8b instruct	0.621	0.359	0.241	0.066
GPT OSS 20b	0.613	0.235	0.138	0.006
Gemma 3 27b it	0.609	0.327	0.264	0.042
Llama 3.2 3b instruct	0.504	0.289	0.184	0.090
Qwen 3 4b	0.794	0.424	0.195	0.090

Interestingly, it was Qwen3's 4b and 8b models that took the lead in this metric, even though they had struggled with execution accuracy in the metric before, while Qwen's 2.5b model was the only one to break 10% exact match accuracy in the hardest question difficulty category.

Partial Matching Accuracy

As a finer grained measure of model success, we also recorded partial matching accuracy on different query components. While the test-suite-sql-eval Python tool splits several of these components into subcategories (e.g. WHERE, WHERE with no operator, where with no HAVING, etc), we aggregated these metrics together for a unified analysis of clause by clause performance.

Table 1: Partial Matching Accuracy: Basic SQL Components

Model	Select	Where	Group
LLaMA 3.1 8B	0.896	0.654	0.806
LLaMA 3.2 3B	0.867	0.619	0.746
Qwen2.5 7B	0.918	0.676	0.849
Qwen 3 4B	0.937	0.789	0.820
Qwen 3 8B	0.926	0.826	0.932
Gemma 3 27B	0.854	0.734	0.875
DeepSeek R1 14B	0.863	0.701	0.815
GPT-OSS 20B	0.960	0.831	1.000

Table 2: Partial Matching Accuracy: Advanced SQL Components

Model	Order	And/Or	Keywords
LLaMA 3.1 8B	0.722	0.957	0.820
LLaMA 3.2 3B	0.585	0.943	0.797
Qwen2.5 7B	0.562	0.950	0.836
Qwen 3 4B	0.958	0.958	0.887
Qwen 3 8B	0.931	0.944	0.894
Gemma 3 27B	0.734	0.947	0.870
DeepSeek R1 14B	0.886	0.943	0.832
GPT-OSS 20B	0.938	0.928	0.943

The partial matching results reveal significant variation in performance across components. For example, GPT-OSS 20b achieved perfect accuracy on GROUP operations, AND/OR operations showed consistently high performance across all models as well, but the WHERE and ORDER BY clauses were the major pain point for several of these models.

Token efficiency

While the Novita API does provide a token usage metric per API call, some models measure only output tokens while others measure total token use. This means that even though our token usage graph and stored results in repo 5 [18] show a hugely disproportinate mean token use per question for different models, this metric does not reflect the actual token consumption per call, and cannot be used to assess models' efficiency on different local systems.

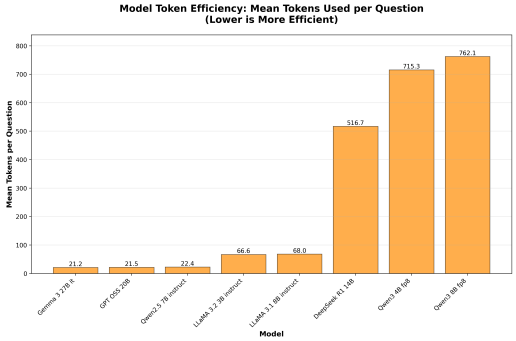

Figure 5: Bar chart of mean token use per call for different models assessed in this paper. Not very helpful.

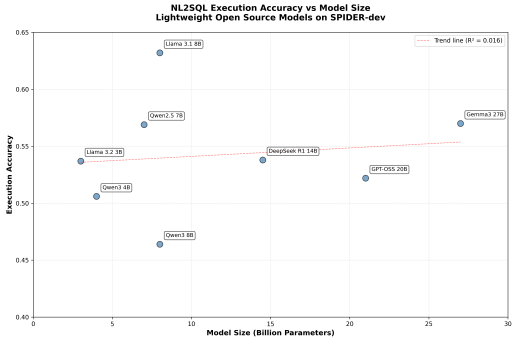

Figure 6: Scatterplot of parameter counts for models vs their asessed performance

Additionally, while it would be possible to measure per call response times, this would not be a meaningful metric to anyone looking to assess these models in time efficiency terms. This is because between unknown exact hardware on the inference API, network speeds, rate limiting, and other factors, there would have been too many confounding factors to include response times as a meaningful metric in our results.

Given then the metrics recorded, these results indicate that while no model was perfect in its ability to produce accurate NL2SQL responses for all of the SPIDER-dev questions and performed significantly worse than known baselines [10], each model nonetheless carries its own strengths and weaknesses. We saw a wide array of performance effectiveness with both execution and exact match accuracy, as well as different distributions of accuracy across different question difficulty levels.

5 ANALYSIS

From the results of our experiment, we can make a few important observations about how smaller models work for the given NL2SQL tasks. Since there is a significant amount of diversity within the set of LLMs that we have chosen to assess, some analysis can be made as to the impact of their design and architecture, and conjecture made as to how these can allow or constrain furthur research in this domain going forward.

The Impact of Model Size And Architecture

While intuitively, LLMs may seem like a domain where performance can be attributed overwhelmingly to parameter count, an actual analysis of the models indicates that parameter count does not affect performance as much as other factors must.

Author's Note - GPT-OSS-20b is actually a 21b parameter model, hence its place on the scatterplot.

In this paper, we review several different models of varying parameter counts. This raises the question of what impact the parameter count of a model has on its performance with the SPIDER-dev test set. The correlation between model size and general performance is significantly more minor than expected (r = 0.019).

Notably, the best and worst performing models in our dataset were both 8b parameter models. This indicates that while parameter count could be important, training modalities and the different training inputs that these models were built on is likely more significant.

However, this also raises the question of how well different model architectures work with these tasks as well. Both instruction tuned models evaluated (the Llama models) performed well, with Llama-3b-instruct performing massively well proportional to its small footprint. However, Deepseek-R1-14b did not perform above expectations, despite being distilled from a significantly more powerful model. While this is not a large sample size, it may indicate that improved comprehension on the input side may be key to unlocking better performance on NL2SQL tasks using this type of small, opensource LLM.

Implications for Model Usefulness

To be blunt, none of these models worked acceptably well as is to be used reliably as a NL2SQL tool, at least not without an immense amount of secondary work beforehand. The best performing contender, while having a 60+% success rate in execution accuracy across all categories of question nonetheless failed to execute the most difficult SPIDER-dev questions 60+% of the time. Furthermore, SPIDER contains far less complex questions than its contemporaries such as SPIDER2 or BIRD.

For a user intending to run a general purpose LLM on their local system without further fine-tuning or configuration, it is still necessary to use a far more powerful model if NL2SQL tasks are part of a commonly required workflow.

Future Work

While this paper does not demonstrate the usefulness of small LLMs in completing NL2SQL tasks in a zeroshot manner, it is not improbable that these models can be a useful tool for text to query

generation. Given the fact that most models tested in this paper succeeded in over 50% of SPIDER-dev test cases, there is potential for future work in this domain.

One avenue of potential improvement in model performance is fine tuning using SPIDER's own data set. Each model could be fine tuned based off of prebuilt prompts and the output evaluated against the "gold SQL". The hope with this strategy would be that having been trained on not just SQL generation examples but generation examples with the exact SQL dialect that would need generation, the model would be able to more reliably generate SQL in a zero-shot fashion.

However, study would have to be made into the performance tradeoffs of fine-tuning the model for SQL generation on other use cases, like code and text generation or general problem solving. This is especially true if we are interested in asessing models for local, general purpose use cases.

Our zero-shot evaluation revealed significant performance limitations, but few-shot learning approaches could potentially improve results by providing models with relevant examples within their context window. By including a handful of high-quality NL2SQL examples that demonstrate proper query construction for similar database schemas or question types, models could better understand the expected output format and reasoning patterns. This approach would be particularly valuable for addressing the WHERE clause generation difficulties we observed across all models, as concrete examples could help models understand how to parse schema for relevant data and make connections between entity types. The effectiveness of k-shot prompting could vary significantly across model architectures as well, meaning that a more clear group of winners/losers may come out of conducting such an assessment.

The component-specific performance analysis revealed that models struggle most with complex reasoning tasks like WHERE clause construction. Implementing explicit chain-of-thought prompting could help models break down complex NL2SQL tasks into manageable steps, such as:

- schema analysis
- entity identification
- relationship mapping
- incremental query construction
- iterative debugging and self assessment

The current obstacle to doing this is the limited token window on small LLMs. However, there is potential in generating prompts with the LLMs themselves, compressing the query window upon running out of token space.

In addition to these potential approaches, combinations of these should also be tested. Only then would we be able to draw a fuller picture of how these models behave as query generation tools.

Overall, the results of this study point to small LLMs on their own and with simplest case zero-shot prompting being insufficient for query generation from natural language, even in relatively simple test sets like SPIDER-dev. Not only this, but at this scale increasing model size does not seem to be a foolproof method of increasing performance, and neither does distilling a compact model from a larger, more powerful model. While results indicate that the tested fine tuned models perform well and better than some of their larger, untuned counterparts, the sample size in this paper is insufficient to assert this claim.

However, these results do not conclusively indicate that there is no potential in using these models to generate queries, especially for relatively low-stakes applications. The number of both small, locally runnable and opensource models and ways to configure them is only growing, and future work needs to be done in this area.

6 CONCLUSION

The asessment of small, opensource LLMs in NL2SQL tasks is an underexplored domain within LLM applications. While their use in code generation tasks like HumanEval is well documented, there is comparatively less data we saw in preliminary resarch into this area. For this reason, we chose to build a testbench using the SPIDER-dev dataset[19] and its accompanying test-suite-sql-eval tool [21]. Using its asessment standards, we ran 1,034 tests on each of the asessed models with the goal of better understanding how smaller LLMs perform when faced with NL2SQL tasks.

This paper evaluated the zero-shot NL2SQL performance of eight lightweight open-source language models ranging from 3-27 billion parameters on the SPIDER-dev dataset. Our assessment examined execution accuracy, exact match accuracy, and query component level performance to understand the current capabilities and limitations of consumer-deployable models for database query generation tasks.

Overall, we found that while some models reached an execution accuracy rating of 60+%, no model was able to complete the SPIDER-dev test set with a >70% success rate. This performance ceiling suggests that our existing zero-shot approach with lightweight models is insufficient for production NL2SQL applications, despite showing promise in specific areas.

Perhaps the most notable finding of our evaluation was that parameter count does not reliably predict NL2SQL performance. The correlation between model size and execution accuracy was fairly negligible ($r = 0.019$), with both the best and worst performing models having 8 billion parameters. This pokes a hole in the intuitive conclusion regarding how scaling works in terms of LLM scaling, and indicates that architecture or fine-tuning are more critical factors than raw model size.

Component-specific analysis of LLM performance revealed clear patterns in model capabilities. WHERE clause generation consistently emerged as a pain point in query generation across all models, while AND/OR operations showed remarkably consistent high performance. This asymmetry between the two component accuracies suggests that the evaluated models do comprehend the correct usage of basic SQL syntax and logical operators but struggle with conditional reasoning and schema comprehension - these being areas that represent specific areas of future exploration or improvement.

For end-users considering deployment of lightweight LLMs for NL2SQL tasks, our findings do not support the use of small and non-pretrained models for direct use in generating queries. While high query completion rates (98%) demonstrate that even small models can reliably produce SQL text, more muted execution accuracy indicates these models are not suited for use as autonomous query

generators. For the simplest query categories, models did show close to 90% accuracy rates. A hypothetical user who has a high error tolerance use case and only needs the simplest possible queries produced would potentially benefit from these models.

Even though execution and exact match rates in our paper were too low for the majority of use cases, we highlighted a few relatively simple ways that these LLMs could potentially be made more effective, while hopefully preserving their utility as a general purpose AI tool. Fine-tuning or k-shot prompting methods could improve performance in this use case albeit with the tradeoff of greater setup times or tradeoffs in performance in other areas.

The open source LLM field will only continue to expand from here on out. While the performance that we saw was not sufficient for direct deployment, the number of freely available models and ways they can be configured is only growing. Our hope is that in the near future, a known working configuration for small LLMs doing NL2SQL tasks can be deployed locally and used for a wide array of applications without dependence on external SAAS providers and closed source models.

This evaluation provides some concrete baseline metrics for lightweight LLM performance on NL2SQL tasks and identifies promising directions for future development. Continued progress toward reliable, locally-deployable NL2SQL tools will require fine-grained approaches that address the specific weaknesses we uncovered while building upon demonstrated strengths of current models.

REFERENCES

[1] DeepSeek AI. 2025. deepseek/deepseek-r1-distill-qwen-14b. https://huggingface.co/deepseek/deepseek-r1-distill-qwen-14b. As of Dec 2025.

[2] Novita AI. 2025. Novita AI. https://novita.ai/. As of Dec 2025.

[3] Alibaba Cloud. 2025. qwen/qwen2.5-7b-instruct. https://huggingface.co/qwen/qwen2.5-7b-instruct. As of Dec 2025.

[4] Alibaba Cloud. 2025. qwen/qwen3-4b-fp8. https://huggingface.co/qwen/qwen3-4b-fp8. As of Dec 2025.

[5] Alibaba Cloud. 2025. qwen/qwen3-8b-fp8. https://huggingface.co/qwen/qwen3-8b-fp8. As of Dec 2025.

[6] A. Corradini, F. Martino, and P. Rossi. 2025. A Comprehensive Survey of Small Language Models in the Era of Large Language Models. *Big Data and Cognitive Computing* 9, 7 (2025), 189. https://doi.org/10.3390/bdcc9070189

[7] Google. 2025. google/gemma-3-27b-it. https://huggingface.co/google/gemma-3-27b-it. As of Dec 2025.

[8] T. Hasan, Y. Li, and M. Chen. 2025. Assessing Small Language Models for Code Generation. *arXiv preprint arXiv:2507.03160* preprint (2025), 1 – 17. https://arxiv.org/abs/2507.03160

[9] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. arXiv:2411.07763 [cs.CL] https://arxiv.org/abs/2411.07763

[10] Boyan Li, Yuyu Luo, Nan Tang, Guoliang Li, and Chengliang Chai. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? *VLDB* 17, 11 (2024), 3318 – 3331. https://www.vldb.org/pvldb/vol17/p3318-luo.pdf

[11] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. arXiv:2305.03111 [cs.CL] https://arxiv.org/abs/2305.03111

[12] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2024. A Survey of Text-to-SQL in the Era of LLMs: Where are we, and where are we going? *arXiv preprint arXiv:2408.05109* 18, 9 (2024), 1 – 20. https://doi.org/10.48550/arXiv.2408.05109

[13] llm stats.com. 2025. LLM Leaderboard. https://llm-stats.com/. As of Dec 2025.

[14] Meta. 2025. meta-llama/llama-3.1-8b-instruct. https://huggingface.co/meta-llama/llama-3.1-8b-instruct. As of Dec 2025.

[15] Meta. 2025. meta-llama/llama-3.2-3b-instruct. https://huggingface.co/meta-llama/llama-3.2-3b-instruct. As of Dec 2025.

[16] OpenAI. 2025. openai/gpt-oss-20b. https://huggingface.co/openai/gpt-oss-20b. As of Dec 2025.

[17] Wenqi Pei, Hailing Xu, Hengyuan Zhao, Shizheng Hou, Han Chen, Zining Zhang, Pingyi Luo, and Bingsheng He. 2025. Feather-SQL: A Lightweight NL2SQL Framework with Dual-Model Collaboration Paradigm for Small Language Models. *arXiv* preprint, 17811 (2025), 1 – 20. https://arxiv.org/abs/2503.17811

[18] GitHub Repository. 2025. Source Code for Experiment Report. https://github.com/CATBOYCOWBOY/comp5118Final.git.

[19] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium.

[20] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2019. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. arXiv:1809.08887 [cs.CL] https://arxiv.org/abs/1809.08887

[21] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suite. In *The 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.